ESTIMATIVA E MÉTRICAS SOFTWARE

AULA 1

Profª Rosemari Pavan Rattmann

CONVERSA INICIAL

Em um mundo cada vez mais digitalizado, as organizações e empresas precisam pensar no software como um investimento estratégico. Não se trata apenas do custo inicial de desenvolvimento, mas também dos gastos contínuos de manutenção e atualização. Essa nova abordagem nos leva a um questionamento crucial: como podemos avaliar e medir adequadamente o valor do software?

Nesse contexto, entram em cena as métricas de software. Elas nos oferecem um arsenal de ferramentas e técnicas para analisar diferentes facetas do desenvolvimento e desempenho do software.

Ao iniciarmos esta jornada de aprendizado, vamos explorar os conceitos fundamentais por trás da medição de software e os tipos mais comuns de métricas. Vamos descobrir também que essas métricas não se limitam ao desenvolvimento de software, podendo ser aplicadas em diversos contextos, como gestão de projetos e análise organizacional.

Além disso, vamos aprender quando e como empregar diferentes métodos de medição de software, e como escolher as métricas mais pertinentes para cada situação. Vamos explorar a importância da estimativa de software, compreendendo como o levantamento de esforço pode moldar o planejamento e a execução de projetos de desenvolvimento.

Por fim, vamos refletir se seria possível viver sem medição de software e quais seriam as implicações de tomar decisões sem base em dados concretos e análises precisas.

Ao término desta etapa, almejamos não apenas uma compreensão mais profunda da relevância da medição de software, mas também a aquisição de habilidades necessárias para aplicar esses conceitos de maneira efetiva em nossa prática profissional.

Vamos iniciar?

TEMA 1 – INTRODUÇÃO AOS CONCEITOS DE MÉTRICAS E ESTIMATIVA DE SOFTWARE

Nesta etapa, vamos explorar os conceitos de métricas e estimativas, dois elementos fundamentais em diversas áreas, desde gestão de projetos até

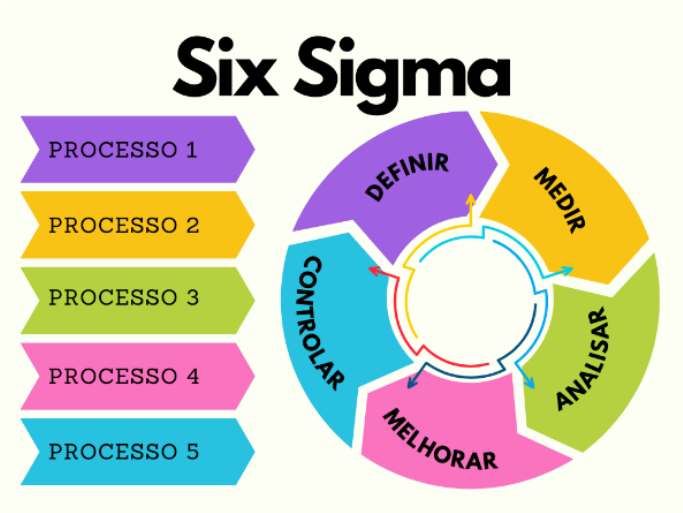
análise de dados. Tais conceitos são essenciais para entendermos como medir e prever resultados em diferentes contextos.

As medidas orientam o progresso da ciência e da engenharia, pois através delas as hipóteses podem ser testadas e validadas empiricamente (De Souza et al., 2003). Isto significa que, quando falamos de métricas, medidas, não estamos nos atendo apenas ao assunto de software. No mundo real, em várias áreas, existem categorias de medidas como as estatísticas, que são muito conhecidas. Por exemplo, quando avaliamos a população de um determinado local em relação à quantidade de homens e mulheres: esse tipo de medida sumariza os valores observados para as métricas em escalas, auxiliando a validação dos modelos e dados analisados. Há muitas categorias estudadas e utilizadas.

Primeiramente, temos estatísticas:

- *Ratio*
- Estatística
- Proporção
- Porcentagem
- Taxa
- Six Sigma (Figura 1)

Figura 1 – Ferramenta para melhorar processos internos



Uma estratégia estatística bastante adotada para melhorar a qualidade e a eficiência em diversas áreas, como o desenvolvimento de software, é o Six Sigma. Essa metodologia foca na diminuição de falhas e inconsistências nos processos, empregando ferramentas estatísticas e uma sequência estruturada de etapas chamada DMAIC: Definir, Medir, Analisar, Melhorar e Controlar. Esses

cinco estágios são cruciais para detectar problemas, implementar soluções eficazes e garantir a manutenção da qualidade, tanto nos processos quanto nos produtos de software. A Figura 1 ilustra essa abordagem, detalhando como cada etapa do DMAIC contribui para a melhoria contínua e a excelência operacional no desenvolvimento de software.

As medidas tomadas são utilizadas para avaliar a consistência de conjuntos de informação, avaliando se os valores são confiáveis em relação aos desvios e erros obtidos. A validação das métricas leva em consideração fórmulas relacionadas à quantidade de erros sistemáticos e erros aleatórios, modelos matemáticos, entre outras formas. Como exemplo, citamos a seguinte fórmula:

- **M = T + s + e**
- **s** (erro sistemático): associado com validade da medida
- **e** (erro aleatório): representa a confiabilidade

Para garantir a confiabilidade de métricas, o método mais fácil é o de teste e reteste. São tomadas duas medidas e calcula-se a correlação entre elas (De Souza et al., 2003). O **teste e reteste** é um método simples para verificar a confiabilidade de uma métrica. No contexto de validação de métricas, ele funciona assim: a mesma métrica é medida duas vezes em condições semelhantes. Depois, calcula-se a correlação entre as duas medições. Se a correlação for alta, isso indica que a métrica é confiável, ou seja, que ela produz resultados consistentes.

No exemplo dado, a fórmula **M = T + s + e** é usada para representar a métrica (**M**), em que **s** é o erro sistemático (ligado à validade da medida) e **e** é o erro aleatório (ligado à confiabilidade). O teste e re-teste ajuda a avaliar o erro aleatório, confirmando se a métrica é consistente ao longo do tempo.

Voltando para o campo do software, as métricas são importantes no desenvolvimento de software, oferecendo dados quantitativos para avaliar qualidade, desempenho e eficácia. Elas abrangem diferentes aspectos do software, como desempenho, tamanho e complexidade, fornecendo uma visão objetiva do progresso do projeto. Essas medidas são essenciais para gestores e desenvolvedores tomarem decisões embasadas, desde o planejamento até a identificação de oportunidades de melhoria.

Sem métricas, compreender o andamento do projeto e identificar problemas pode ser desafiador, levando a decisões subjetivas e arriscadas, além

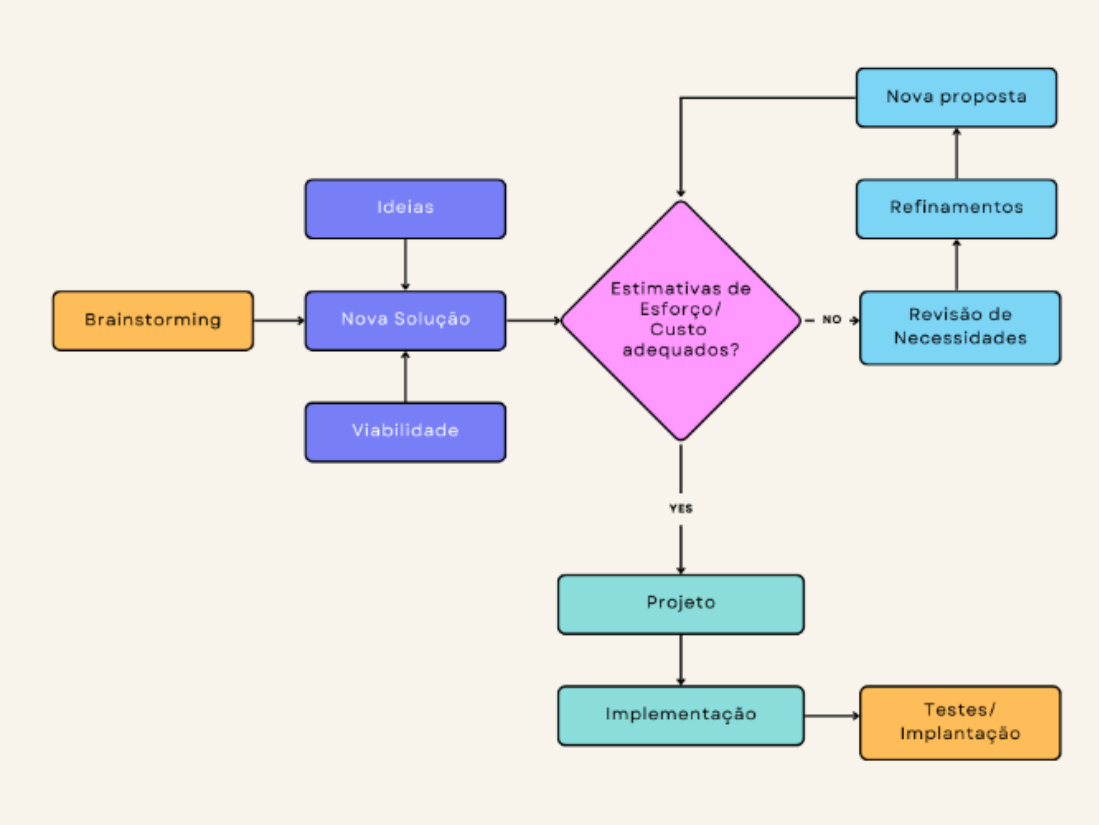
de dificultar a detecção precoce de problemas que possam afetar a qualidade ou desempenho do software.

No entanto, é importante escolher as métricas corretas e garantir que elas estejam alinhadas com os objetivos e requisitos específicos do projeto ou processo em questão. Métricas mal definidas ou mal interpretadas podem levar a conclusões equivocadas ou ações inadequadas.

As estimativas referem-se ao processo de previsão ou cálculo de valores futuros, com base em informações disponíveis no presente. Na prática, as estimativas são utilizadas para antecipar recursos necessários, tempo de conclusão de uma tarefa ou projeto, custos envolvidos, entre outros aspectos. Muitas vezes, as métricas direcionam as decisões sobre compra de novos produtos, o desenvolvimento interno ou o cancelamento de propostas, como exemplificado pela Figura 2.

Existem várias abordagens para realizar estimativas, incluindo técnicas determinísticas, como análise de pontos de função ou estimativa *bottom-up*, além de técnicas probabilísticas, como simulação de Monte Carlo ou análise de risco.

Figura 2 – Fluxograma para viabilidade de uma nova proposta



Outras formas de medição podem ser aplicadas no projeto para avaliar a qualidade da construção do software, como as medições baseadas em coesão e acoplamento (Yourdon, 2004). Em desenvolvimento de software, o acoplamento refere-se à medida de dependência entre módulos ou componentes. Quanto menor o acoplamento, menos um componente depende de outros. Coesão, por outro lado, indica o grau em que os elementos de um módulo estão relacionados e trabalham juntos para realizar uma única tarefa ou responsabilidade. Idealmente, busca-se alto nível de coesão e baixo acoplamento para garantir que o código seja modular, sendo fácil de manter e modificar.

É importante ressaltar que as estimativas estão sujeitas a incertezas e variabilidades, devendo ser constantemente revisadas e ajustadas à medida que mais informações se tornam disponíveis ao longo do tempo. Além disso, a precisão das estimativas pode ser influenciada por fatores como a experiência da equipe, a complexidade do projeto e a disponibilidade de dados históricos relevantes.

Vamos considerar um exemplo prático para ilustrar a correlação entre métricas e estimativas. Imagine que uma equipe de desenvolvimento de software esteja trabalhando em um projeto para criar um aplicativo móvel. Para monitorar o progresso do desenvolvimento e avaliar a qualidade do software, a equipe utiliza métricas como o número de bugs identificados por semana, a taxa de cobertura de testes e o tempo médio de resposta do aplicativo.

Com base nessas métricas e em outras informações disponíveis, a equipe precisa fazer uma estimativa do tempo necessário para concluir o desenvolvimento do aplicativo. Para isso, eles consideram fatores como a quantidade de funcionalidades restantes para implementar, a produtividade da equipe e eventuais imprevistos.

Assim, as métricas fornecem dados concretos sobre a qualidade e o desempenho do software durante o processo de desenvolvimento. Por exemplo, se a equipe observar um aumento repentino no número de bugs identificados, isso pode indicar problemas na qualidade do código ou nos processos de teste que precisam ser abordados.

Por sua vez, as estimativas são utilizadas para prever o tempo necessário para completar o projeto com base nessas métricas e em outras informações disponíveis. Por exemplo, se a equipe identificar um aumento significativo na

taxa de defeitos encontrados, eles podem ajustar suas estimativas de tempo para acomodar o trabalho adicional necessário para corrigir esses problemas.

Dessa forma, as métricas e as estimativas trabalham juntas para fornecer uma visão abrangente do projeto de desenvolvimento de software, permitindo à equipe tomar decisões embasadas em dados e planejar adequadamente o tempo e os recursos necessários para concluir o projeto com sucesso.

1.1 Métricas

- Métricas são medidas quantitativas utilizadas para avaliar e quantificar características específicas de um processo, sistema ou fenômeno, fornecendo dados objetivos e mensuráveis para análise e tomada de decisão.
- Métricas são parâmetros mensuráveis utilizados para quantificar aspectos como desempenho, qualidade, eficiência ou complexidade de um sistema ou processo, fornecendo uma base sólida para a análise e tomada de decisões.
- Métricas são parâmetros quantitativos utilizados para mensurar e monitorar o desempenho e a eficácia de processos organizacionais, permitindo a identificação de pontos fortes e áreas de melhoria em empresas de diversos setores, como manufatura, finanças e varejo (Santos, 2020).

1.1.1 Tipos de métricas

Existem várias categorias de métricas que podem ser aplicadas em diversos contextos além do software (Figura 3). Elas incluem:

- **Métricas de desempenho**: são medidas quantitativas utilizadas para avaliar e monitorar a eficiência e eficácia de sistemas, processos ou atividades em diversas áreas. Elas fornecem uma base objetiva para a análise do desempenho organizacional e individual, permitindo a identificação de pontos fortes, áreas de melhoria e oportunidades de otimização. O artigo explora a definição, aplicação e importância das métricas de desempenho, oferecendo insights sobre como elas podem ser

utilizadas para impulsionar o sucesso e a competitividade em diferentes contextos (Guerreiro, 2021).

- **Métricas de qualidade**: as métricas de qualidade referem-se a medidas quantitativas utilizadas para avaliar e monitorar a conformidade com padrões de qualidade definidos e a satisfação do usuário com produtos ou serviços. Elas fornecem indicadores objetivos sobre a qualidade de um produto ou processo, permitindo uma avaliação sistemática e fundamentada. Existem diferentes tipos de métricas de qualidade, incluindo métricas de qualidade de produto, que avaliam características intrínsecas de um produto ou serviço, como confiabilidade, usabilidade e desempenho; e métricas de qualidade de processo, que mensuram a eficácia e eficiência dos processos utilizados para desenvolver ou fornecer um produto ou serviço (Guerreiro, 2021).
- **Métricas de produtividade**: são indicadores quantitativos utilizados para medir a relação entre input e output em um processo ou atividade. Elas permitem avaliar a eficiência e eficácia de um processo, identificar oportunidades de melhoria e acompanhar o desempenho ao longo do tempo. Este artigo visa fornecer uma visão abrangente sobre métricas de produtividade, destacando sua importância e aplicação em diversos contextos (Frezatti, 2020).
- **Métricas de impacto**: indicadores quantitativos e qualitativos que avaliam o efeito ou a influência de uma ação, intervenção ou projeto em um determinado contexto. Elas fornecem informações sobre os resultados alcançados e o valor agregado por uma iniciativa, permitindo a análise de sua eficácia e efetividade. Esta etapa visa explorar o conceito de métricas de impacto, sua importância e aplicação em diversos campos, como pesquisa, políticas públicas, desenvolvimento social e ambiental (Frezatti, 2020).

Figura 3 – Tipos de métricas

1.1.2 Características das métricas

- Devem ser mensuráveis e quantificáveis.
- Devem ser relevantes para os objetivos do projeto ou processo.
- Devem ser consistentes e confiáveis ao longo do tempo.
- Podem ser expressas em diferentes unidades, dependendo do contexto.

1.1.3 Exemplos de métricas

Métricas de desempenho:

- Tempo de resposta do sistema: mede o tempo necessário para o sistema responder a uma solicitação do usuário.
- Taxa de disponibilidade: indica a proporção de tempo em que o sistema está disponível para uso em relação ao tempo total.
- Taxa de erro: quantifica a proporção de erros ou falhas que ocorrem durante a execução do sistema.

Figura 4 – Métricas de desempenho ajudam na tomada de decisões

Crédito: OneLineStock/Shutterstock.

Métricas de qualidade:

- Taxa de defeitos por linha de código: calcula a quantidade de defeitos encontrados em relação ao número de linhas de código.
- Nível de satisfação do cliente: avalia a percepção do cliente em relação à qualidade do produto ou serviço oferecido.
- Conformidade com padrões: mensura o grau em que o produto ou processo atende aos padrões e requisitos estabelecidos.

Métricas de produtividade:

- Produção por hora: quantifica a quantidade de produtos ou serviços entregues por hora de trabalho.
- Taxa de utilização de recursos: mede a eficiência na utilização dos recursos disponíveis em um processo ou atividade.
- Relação entre input e output: calcula a relação entre os recursos investidos e os resultados obtidos em um determinado período de tempo.

Métricas de impacto:

- Número de vidas salvas: mensura o impacto de um programa ou intervenção na saúde pública ao contar o número de vidas salvas.
- Taxa de empregabilidade: avalia o efeito de programas de capacitação profissional na taxa de empregabilidade dos participantes.
- Redução da emissão de carbono: quantifica o impacto de iniciativas ambientais na redução das emissões de carbono na atmosfera.

1.2 Estimativas

- Estimativas são projeções ou previsões de valores futuros com base em informações disponíveis no presente. Elas fornecem uma avaliação antecipada de recursos necessários, tempo de conclusão de uma tarefa ou projeto, custos envolvidos, entre outros aspectos.
- Estimativas são cálculos ou previsões baseadas em dados disponíveis e suposições razoáveis para determinar uma expectativa de resultado futuro. Elas são amplamente utilizadas em diversos campos, como economia, engenharia, ciências sociais, entre outros, para planejar e tomar decisões informadas (Frezatti, 2020).

1.2.1 Tipos de estimativas

- **Estimativas determinísticas:** são baseadas em dados concretos e precisos, quando há pouca incerteza. São comumente aplicadas no desenvolvimento de software quando os requisitos do projeto são bem definidos, os processos são conhecidos e não há muita incerteza em relação aos recursos necessários. No entanto, elas podem não ser adequadas em projetos complexos ou sujeitos a mudanças frequentes, quando a variabilidade dos dados é alta. Exemplos:
    - **Estimativa de Esforço:** previsão da quantidade de trabalho necessária para completar uma tarefa, projeto ou atividade, sem considerar variações ou incertezas.
    - **Estimativa de tempo:** previsão do tempo necessário para concluir uma tarefa, projeto ou entrega, sem levar em conta incertezas ou variações.
    - **Estimativa de custos:** previsão dos custos envolvidos em um projeto, produto ou serviço, sem considerar variações ou incertezas.
- **Estimativas probabilísticas**: consideram a incerteza e variabilidade, usando técnicas como análise de Monte Carlo, para calcular probabilidades de diferentes resultados. Ao contrário das estimativas determinísticas, as probabilísticas fornecem uma faixa de valores possíveis juntamente com a probabilidade de ocorrência de cada resultado. As estimativas probabilísticas são aplicadas em projetos de

software quando há uma grande incerteza em relação aos requisitos, recursos e outros fatores que podem influenciar o desempenho do projeto. Elas são especialmente úteis em projetos complexos, sujeitos a mudanças frequentes ou com requisitos mal definidos, permitindo uma análise mais realista e abrangente do cronograma e dos recursos necessários. Exemplos:

- **Estimativa de tamanho**: previsão do tamanho de um projeto de *software* em termos de linhas de código, pontos de função ou outras unidades de medida, considerando incertezas e variações.
- **Estimativa de riscos**: avaliação das possíveis incertezas, ameaças ou oportunidades que podem afetar um projeto, produto ou serviço, considerando sua probabilidade e impacto.

O uso adequado de métricas e estimativas é essencial para o sucesso de qualquer projeto, proporcionando uma base sólida para a tomada de decisões, o gerenciamento eficaz de recursos e a entrega de resultados de alta qualidade. No contexto específico de projetos de software, essas práticas são ainda mais críticas, devido à natureza dinâmica e complexa do desenvolvimento de software.

TEMA 2 – IMPORTÂNCIA DAS MÉTRICAS E ESTIMATIVAS NO DESENVOLVIMENTO DE SOFTWARE

2.1 Importância de medir software

Medir o software é essencial para garantir o sucesso de um projeto de desenvolvimento. Imagine construir uma casa sem instrumentos de medição. Seria difícil saber se as paredes estão retas, se o telhado está nivelado ou se o acabamento está correto. Da mesma forma, no desenvolvimento de software, as métricas são como ferramentas que nos permitem avaliar o progresso, a qualidade e a eficiência do trabalho realizado. Sem elas, estaríamos navegando no escuro, sem noção do que está acontecendo e quais problemas podem surgir. Aqui estão alguns pontos que destacam a importância de medir software:

- **Avaliação de progresso:** as métricas nos fornecem uma maneira objetiva de acompanhar o andamento do projeto. Podemos verificar se estamos seguindo o cronograma planejado, se estamos atingindo os

marcos estabelecidos e se há a necessidade de ajustes para manter o rumo certo.

- **Garantia de qualidade:** ao medir a qualidade do software, podemos identificar áreas problemáticas que precisam de atenção. Métricas como taxa de defeitos, cobertura de código e tempo médio entre falhas nos ajudam a manter um padrão de qualidade elevado e a garantir a satisfação do cliente.
- **Otimização de processos**: medir o tempo gasto em diferentes atividades do desenvolvimento nos permite identificar gargalos e oportunidades de melhoria. Podemos encontrar maneiras de otimizar o processo, reduzindo custos, prazos e esforço desnecessário.
- **Tomada de decisão embasada em dados:** com métricas em mãos, podemos tomar decisões informadas sobre prioridades, alocação de recursos e estratégias de desenvolvimento. Isso nos ajuda a evitar decisões baseadas apenas em intuição ou opinião pessoal, reduzindo os riscos e aumentando as chances de sucesso.

A medição de software não é apenas útil, mas fundamental para garantir a entrega de produtos de alta qualidade, dentro do prazo e do orçamento estabelecidos.

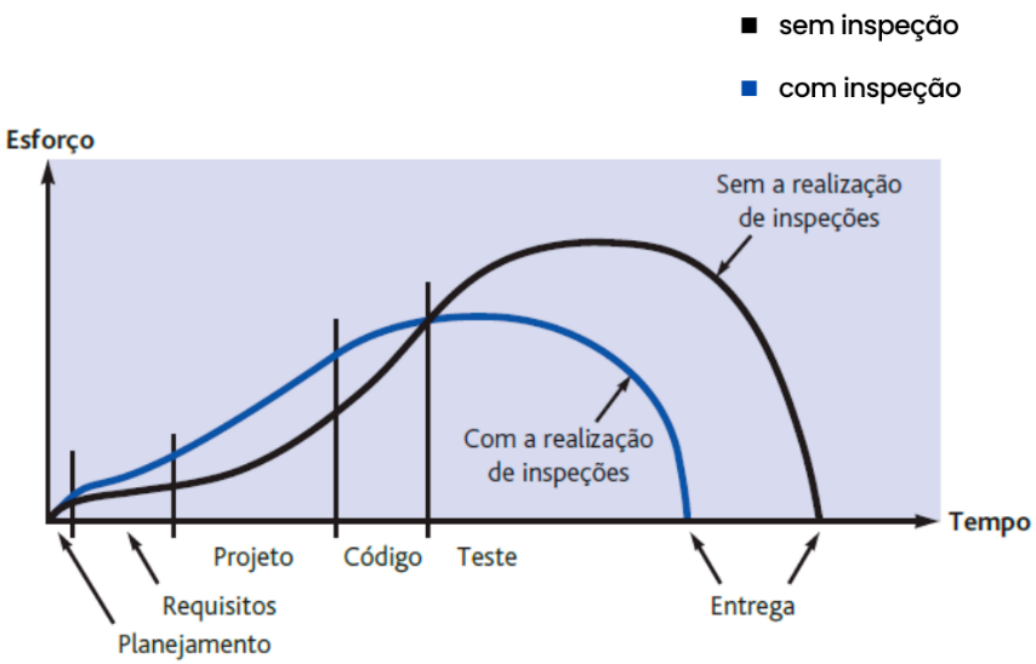
2.2 Importância de medir software em diferentes contextos

Medir software vai além de apenas avaliar o progresso e a qualidade de projetos de desenvolvimento. No contexto da compra de software de terceiros ou soluções prontas, bem como na venda de software e serviços, a medição torna-se ainda mais importante. Vamos explorar a importância dessas métricas em cada um desses cenários, e como elas influenciam o esforço, os custos e o tempo envolvidos.

- **Compra de software**: ao considerar a compra de software pronto, as métricas desempenham um papel fundamental na tomada de decisão. Empresas precisam avaliar não apenas o custo inicial da solução, mas também os custos contínuos de manutenção, atualizações e suporte. Métricas de qualidade, como taxa de defeitos e satisfação do cliente, ajudam a garantir que o software atenda às necessidades e expectativas

da empresa (Pressman, 2016). Por exemplo, imagine uma empresa que está considerando comprar um sistema de gerenciamento de recursos humanos. Ao medir a qualidade do software por meio de métricas como taxa de defeitos e satisfação do cliente, ela pode identificar se a solução atende aos requisitos de funcionamento e desempenho esperados. Além disso, métricas de desempenho, como tempo médio de resposta e escalabilidade, são essenciais para garantir que o software possa lidar com a carga de trabalho da empresa. Para exemplificar essa situação, vemos um gráfico na Figura 5 sobre uma curva de entrega de produto que teve um plano de inspeção estimado com antecedência e o quanto ele reduz, em comparação com outro plano que não levou em conta as inspeções antecipadas. A estimativa de software, as métricas iniciais são interessantes para reduzir riscos.

Figura 5 – Esforço despendido com e sem o emprego de revisões antecipadas



Fonte: Pressman, 2016.

- **Venda de software e serviços:** as métricas desempenham um papel duplo: ajudam os fornecedores a entenderem as necessidades e expectativas dos clientes, ao mesmo tempo em que garantem que os serviços sejam entregues de maneira eficiente e lucrativa. Métricas como

esforço homem/hora e custos são fundamentais para determinar os preços dos serviços e garantir que sejam lucrativos (Sommerville, 2016). Por exemplo, uma empresa de desenvolvimento de software que oferece serviços de personalização de sistemas para clientes corporativos precisa medir o esforço homem/hora necessário para cada projeto. Isso inclui o tempo gasto na análise de requisitos, no desenvolvimento de código, nos testes e na implantação. Com base nessas métricas, a empresa pode determinar um preço justo para seus serviços, levando em consideração não apenas os custos diretos, mas também os custos indiretos, como salários, benefícios e despesas gerais.

- **Tempo**: é um dos recursos mais preciosos no desenvolvimento de software, e sua medição é crucial para garantir o sucesso do projeto. O tempo deve ser medido não apenas em termos de horas trabalhadas, mas também em relação à importância do software para o negócio. Projetos críticos, que impactam diretamente a operação da empresa ou a satisfação do cliente, devem receber maior atenção e prioridade (Pfleeger, 2016).

Imagine uma empresa de comércio eletrônico que vai lançar uma nova plataforma de vendas online. O tempo de desenvolvimento e implementação dessa plataforma é essencial para garantir que a empresa não perca oportunidades de negócio durante períodos de alta demanda, como as festas de fim de ano. Nesse caso, medir o tempo de desenvolvimento de forma eficiente e garantir que o software seja entregue no prazo é crucial para o sucesso do projeto e para a reputação da empresa.

Métricas de qualidade, esforço e tempo são essenciais para garantir que os projetos sejam entregues dentro do prazo, dentro do orçamento e com a qualidade esperada pelos clientes.

2.3 Medição de software e possibilidades de medições

Com o avanço da tecnologia, surgiram diversas possibilidades modernas de medição que oferecem insights cada vez mais detalhados e precisos. Vamos explorar o conceito de medição de software e as diversas opções disponíveis, com exemplos e argumentos para ilustrar sua importância e utilidade.

- **Análise estática de código:** a análise estática de código se tornou uma ferramenta indispensável para os desenvolvedores modernos. Ferramentas como SonarQube e ESLint permitem analisar o código-fonte em busca de padrões de codificação inconsistentes, vulnerabilidades de segurança e outros problemas que podem afetar a qualidade e manutenibilidade do software. Exemplo: ao usar o SonarQube em um projeto de desenvolvimento de software, a equipe pode identificar áreas de código com alta complexidade ou violações de boas práticas de codificação e corrigir esses problemas antes que eles se tornem fontes de bugs ou problemas de segurança (Pressman, 2016).
- **Métricas de desempenho de aplicativos:** com o aumento da demanda por aplicativos de alto desempenho, métricas como tempo de resposta, taxa de erro e uso de recursos se tornaram críticas. Ferramentas como o New Relic e o Datadog permitem monitorar o desempenho do aplicativo em tempo real e identificar possíveis gargalos ou problemas de escalabilidade. Exemplo: uma empresa de comércio eletrônico pode usar o New Relic para monitorar o desempenho de seu aplicativo móvel durante um evento de vendas. Se o tempo de resposta começar a ficar lento devido ao aumento do tráfego, a equipe pode identificar e corrigir o problema rapidamente, para evitar impactos negativos na experiência do usuário (Sommerville, 2016).
- **Análise de dados de usuários:** a análise de dados de usuários se tornou uma parte essencial do desenvolvimento de software moderno. Ferramentas como o Google Analytics e o Mixpanel permitem coletar e analisar dados sobre o comportamento dos usuários, incluindo taxa de conversão, tempo de engajamento e retenção de usuários. Exemplo: uma empresa de software pode usar o Google Analytics para acompanhar como os usuários interagem com seu aplicativo, quais recursos são mais populares e onde os usuários estão encontrando dificuldades. Com essas informações, a equipe pode ajustar a interface do usuário e as funcionalidades do aplicativo para melhor atender às necessidades dos usuários (Pfleeger, 2016).

Com as possibilidades modernas de medição, as equipes de desenvolvimento têm acesso a uma variedade de ferramentas e técnicas poderosas para avaliar e melhorar seus projetos de software. Ao adotar essas

práticas de medição, as equipes podem garantir que seus projetos atendam às expectativas dos clientes e se destaquem em um mercado cada vez mais competitivo.

2.4 Tipos de medição de software e principais ferramentas

Existem diversos tipos de medição de software, cada um focado em aspectos específicos do desenvolvimento:

- **Medição de tamanho**: um dos tipos mais básicos de medição de software é a medição de tamanho, que envolve quantificar o tamanho do software em termos de linhas de código, número de arquivos ou outras unidades de medida. A medição de tamanho é útil para avaliar a complexidade e escala de um projeto de software.
- **CLOC (Count Lines of Code):** é uma ferramenta de linha de comando que conta o número de linhas de código em um projeto de software. Ele é amplamente utilizado para calcular a quantidade de código-fonte em linguagens de programação diversas, como C, C++, Java e Python (Figura 6).

Figura 6 – Exemplo sobre contagem de linhas de código

Language	files	blank	comment	code
PHP	715	41385	120565	187569
JavaScript	361	19528	27389	111154
CSS	202	15586	3655	77204
Sass	10	154	5	529
Markdown	2	129	0	241
HTML	1	13	0	85
XML	1	6	0	37
SUM:	1292	76801	151614	376819

- **SLOCCount:** é outra ferramenta de linha de comando que calcula o número de linhas de código em um projeto, mas também fornece outras métricas, como o número de arquivos e o número de funções. Ele suporta uma variedade de linguagens de programação e é útil para avaliar a complexidade de projetos grandes e complexos.

Exemplo: imagine uma equipe de desenvolvimento que está trabalhando em um projeto de software para uma empresa de serviços financeiros. Eles usam

o CLOC para contar o número de linhas de código em seu projeto e descobrem que o software tem mais de 50.000 linhas de código. Isso os ajuda a entender a escala do projeto e planejar adequadamente o trabalho de desenvolvimento (Pressman, 2016).

- **Medição de complexidade**: a medição de complexidade envolve avaliar o quão complexo é o software em termos de sua estrutura, fluxo de controle e dependências entre componentes. Isso ajuda a identificar áreas de código que podem ser difíceis de entender, manter ou testar.
- **McCabe IQ**: é uma ferramenta de análise estática de código que calcula métricas de complexidade, como o número de caminhos de controle e o fator de complexidade ciclomática. Ele ajuda os desenvolvedores a identificarem áreas de código com complexidade excessiva e tomar medidas para simplificá-las.
- **SonarQube:** é uma plataforma de análise de código-fonte que fornece uma variedade de métricas de qualidade de código, incluindo métricas de complexidade. Ele identifica padrões de codificação complexos e fornece recomendações para melhorar a qualidade do código.

Exemplo: uma equipe de desenvolvimento está revisando o código-fonte de um aplicativo web usando o SonarQube. Eles descobrem que uma função específica tem uma complexidade ciclomática muito alta, o que indica que ela pode ser difícil de entender e testar. Eles decidem refatorar a função para reduzir sua complexidade e melhorar a qualidade do código (Sommerville, 2016).

- **Medição de desempenho**: a medição de desempenho envolve avaliar o desempenho do software em termos de tempo de resposta, taxa de transferência e consumo de recursos. Isso é crucial para garantir que o software atenda aos requisitos de desempenho e escalabilidade A Figura 7 ajuda a avaliar o desempenho do software, considerando tempo de resposta, taxa de transferência, e consumo de recursos, essenciais para garantir que o software atenda aos requisitos de desempenho e escalabilidade.
    - Quadrante superior direito ("Faz o certo do jeito certo"): o software atinge seus objetivos de forma eficaz e utiliza os recursos de maneira eficiente, representando o cenário ideal.

- Quadrante superior esquerdo ("Faz o errado do jeito certo"): o software é eficiente no uso dos recursos, mas falha em cumprir os objetivos corretos.
- Quadrante inferior esquerdo ("Faz o errado do jeito errado"): o software não atinge os objetivos e ainda desperdiça recursos, sendo o pior cenário.
- Quadrante inferior direito ("Faz o certo do jeito errado"): o software cumpre seus objetivos, mas de maneira ineficiente, utilizando mais recursos do que o necessário.

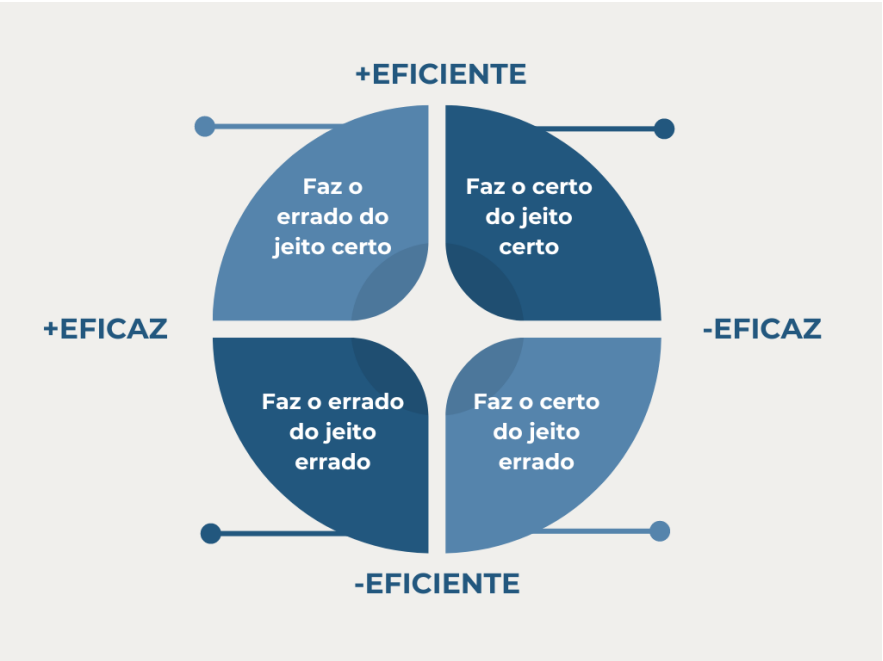


Figura 7 – Exemplo de medida de desempenho

- **JMeter**: é uma ferramenta de teste de desempenho que permite simular cargas de trabalho de usuários em aplicativos web e medir o desempenho do servidor. Ele fornece métricas detalhadas de desempenho, como tempo de resposta, taxa de erro e uso de recursos.
- **Apache Bench**: é uma ferramenta de linha de comando que realiza testes de benchmarking em servidores web, medindo o desempenho em termos de tempo de resposta e taxa de transferência. É útil para avaliar o desempenho de servidores web sob carga.

Exemplo: uma equipe de desenvolvimento está testando o desempenho de um aplicativo de comércio eletrônico usando o JMeter. Eles simulam uma

carga de trabalho de milhares de usuários acessando o site ao mesmo tempo e monitoram o tempo de resposta do servidor. Com base nos resultados dos testes, eles identificam gargalos de desempenho e otimizam o código para melhorar a escalabilidade do aplicativo (Pfleeger, 2016).

2.5 Fase de medição e fase de estimativa

O desenvolvimento de software é um processo complexo que envolve diversas etapas, desde a concepção da ideia até a entrega do produto final. Duas fases fundamentais desse processo são a fase de medição e a fase de estimativa, que desempenham papéis essenciais na garantia da qualidade, no planejamento e na execução dos projetos de software.

- **Fase de medição:** os desenvolvedores utilizam métricas específicas para quantificar características do software, como tamanho, complexidade, qualidade e desempenho. Essas métricas fornecem informações objetivas que ajudam a monitorar o progresso do projeto, identificar possíveis problemas e avaliar a eficácia das estratégias adotadas.

A fase de medição é importante para garantir que o software atenda aos requisitos de qualidade e desempenho estabelecidos. Ao monitorar de perto as métricas relevantes, os desenvolvedores podem identificar problemas precocemente e tomar medidas corretivas antes que eles se tornem mais difíceis e custosos de resolver.

- **Fase de estimativa:** na fase de estimativa, os desenvolvedores utilizam técnicas e métodos para prever o esforço, o tempo e os recursos necessários para concluir um projeto de software. Essa fase é crucial para o planejamento e o gerenciamento eficaz do projeto, permitindo que as equipes definam metas realistas e aloquem recursos de forma adequada.

Existem diversas técnicas de estimativa disponíveis. Uma das mais comuns é a estimativa por pontos de função, que quantifica o tamanho funcional do software com base nas funcionalidades que ele oferece aos usuários, como veremos adiante. Outras técnicas, como a estimativa analógica e a estimativa paramétrica, utilizam dados históricos e modelos estatísticos para prever o esforço e o tempo necessários para o desenvolvimento do software.

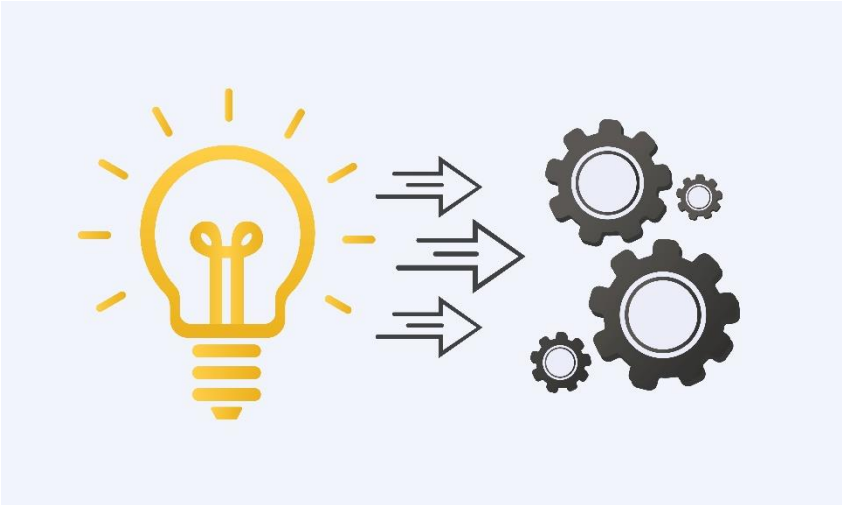
A fase de estimativa é fundamental para o sucesso do projeto, pois ajuda a evitar atrasos, estouros de orçamento e insatisfação do cliente. Ao realizar

estimativas precisas e realistas, as equipes de desenvolvimento podem planejar adequadamente as atividades, definir prazos viáveis e tomar decisões informadas ao longo do ciclo de vida do projeto.

TEMA 3 – VISÃO GERAL DE MÉTRICAS DE SOFTWARE

A medição de software envolve a coleta e a análise de dados relacionados ao processo de desenvolvimento, ao produto final ou aos recursos envolvidos. Por meio da definição de métricas específicas, como tamanho do código, complexidade, esforço de desenvolvimento e qualidade, os desenvolvedores podem obter insights valiosos sobre o progresso e a eficácia do projeto. Geralmente, as empresas criam suas metodologias para configurações e medições, gerando um entendimento padronizado para todos, o que é exemplificado como um plano de ação na figura a seguir.

Figura 8 – Plano de ação para padronizar as metodologias: mente – ideias – configuração e ação



Crédito: ICONMAN66/Shutterstock.

As métricas podem ser coletadas em diferentes estágios do ciclo de vida do software, desde a fase de planejamento até a manutenção pós-implantação. Além disso, a medição pode ser aplicada a diferentes elementos do software, como código-fonte, requisitos, testes e desempenho.

- **Técnicas de medição:** existem várias técnicas de medição de software amplamente utilizadas, cada uma com suas vantagens e aplicações específicas. Entre as técnicas mais comuns estão a contagem de linhas

de código (LOC), a análise estática de código, a análise de pontos de função e a análise de complexidade.

A contagem de linhas de código é uma técnica simples que fornece uma medida do tamanho do software, enquanto a análise estática de código avalia a qualidade e a conformidade do código com padrões definidos. Já a análise de pontos de função é uma técnica mais abstrata, pois mede o tamanho funcional do software com base nas funcionalidades oferecidas aos usuários.

- **Métodos de medição:** os métodos de medição abarcam os procedimentos e abordagens adotados para coletar, analisar e interpretar os dados. Esses métodos podem variar de acordo com o contexto do projeto e as necessidades específicas de medição. Alguns dos métodos mais comuns incluem a medição direta, a medição indireta e a medição baseada em padrões.

Na medição direta, os dados são coletados diretamente do software ou do processo de desenvolvimento, enquanto na medição indireta os dados são inferidos a partir de outras medidas ou indicadores. Já a medição baseada em padrões utiliza benchmarks e referências externas para comparar o desempenho ou a qualidade do software.

3.1 Técnicas de medição

Vamos nos aprofundar nas técnicas específicas utilizadas na medição de software. Essas técnicas fornecem uma maneira estruturada e sistemática de coletar e analisar dados sobre o software, permitindo que os desenvolvedores obtenham insights valiosos sobre seu desempenho e qualidade.

- **Contagem de Linhas de Código (LOC)**: a contagem de linhas de código é uma das técnicas mais antigas e amplamente utilizadas na medição de software. Ela envolve simplesmente contar o número de linhas de código-fonte em um programa ou sistema. Embora seja uma métrica simples, a contagem de LOC fornece uma medida básica do tamanho do software, o que pode ser útil para estimar o esforço de desenvolvimento e avaliar a complexidade do código.
- **Análise estática de código**: a análise estática de código é uma técnica que examina o código-fonte sem executá-lo, identificando padrões de

codificação, erros sintáticos e potenciais problemas de segurança. Ferramentas como o SonarQube e o Checkstyle são comumente utilizadas para realizar análises estáticas de código, ajudando os desenvolvedores a melhorarem a qualidade e a manutenibilidade do software.

- **Análise de pontos de função:** a análise de pontos de função é uma técnica mais abstrata. Ela mede o tamanho funcional do software com base nas funcionalidades oferecidas aos usuários. Ela atribui valores numéricos às funções do software, como entradas, saídas, consultas e arquivos, calculando o tamanho funcional total com base nessas medidas. Essa técnica é útil para estimar o tamanho e o esforço de desenvolvimento de sistemas baseados em requisitos funcionais (Vazquez, 2003). O IFPUG – International Function Point Users Group (Grupo Internacional de Usuários de Pontos de Função) é uma organização global sem fins lucrativos, fundada em 1986, dedicada à padronização e promoção do uso de pontos de função, ajudando organizações em todo o mundo a gerenciarem e medirem projetos de software de maneira mais eficaz.

Figura 9 – IFPUG

3.2 Metodologias para medição

As metodologias fornecem uma estrutura e um conjunto de diretrizes para realizar medições de forma eficaz e consistente, garantindo resultados confiáveis e úteis para os desenvolvedores.

- **Metodologia de medição direta**: envolve a coleta de dados diretamente do software ou do processo de desenvolvimento. Pode incluir a contagem de linhas de código, a análise de métricas de qualidade de código e a coleta de dados de desempenho do sistema em tempo de execução. Essa

metodologia é útil para obter insights específicos sobre o software em si e suas características.

- **Metodologia de medição indireta:** envolve a inferência de dados a partir de outras medidas ou indicadores. Isso pode incluir a análise de métricas relacionadas, como complexidade do código versus taxa de defeitos, para identificar padrões ou tendências. A medição indireta é útil quando os dados diretos não estão disponíveis ou são difíceis de coletar.
- **Metodologia de medição baseada em padrões**: utiliza benchmarks e referências externas para comparar o desempenho ou a qualidade do software. Isso pode incluir a comparação do software em desenvolvimento com outros produtos semelhantes no mercado ou com padrões estabelecidos pela indústria. Essa metodologia é útil para avaliar.

3.3 Exemplos de aplicação de medições

- **Exemplo de Contagem de Linhas de Código (LOC):** estimativa de esforço de desenvolvimento para um projeto de software. Ao contar o número de linhas de código em um sistema semelhante desenvolvido anteriormente, os gerentes de projeto podem fazer uma estimativa aproximada do tempo e dos recursos necessários para desenvolver um novo sistema semelhante.
- **Exemplo de análise estática de código:** identificação de vulnerabilidades de segurança em um aplicativo web. Ferramentas de análise estática, como o OWASP ZAP, podem examinar o código-fonte em busca de possíveis vulnerabilidades, como injeção de SQL, XSS e CSRF, ajudando os desenvolvedores a corrigirem esses problemas antes do lançamento do produto.
- **Exemplo de Análise de Pontos de Função (FPA):** estimativa de tamanho e esforço de desenvolvimento para um novo sistema de gerenciamento de banco de dados. Ao analisar os requisitos funcionais do sistema e atribuir valores de pontos de função às diferentes funcionalidades, os desenvolvedores podem estimar o tamanho do sistema e o esforço necessário para desenvolvê-lo (Vazquez, 2003).

Como já estudamos, as estimativas de software ajudam a definir metas realistas, alocar recursos de forma adequada e tomar decisões informadas ao longo do ciclo de vida do projeto. Vamos avaliar, agora, como funciona essa atividade.

As estimativas de software envolvem a previsão do esforço, do tempo e dos recursos necessários para desenvolver um sistema de software. Existem várias abordagens e técnicas para realizar estimativas de software, cada uma com suas vantagens e desafios.

Algumas das técnicas mais comuns incluem:

- **Estimativa por analogia:** esta técnica envolve a comparação do projeto atual com projetos similares realizados anteriormente. Baseando-se em projetos anteriores, os desenvolvedores podem fazer uma estimativa aproximada do esforço e do tempo necessários para o novo projeto.
- **Estimativa paramétrica:** nesta técnica, modelos matemáticos são utilizados para prever o esforço e o tempo com base em parâmetros específicos do projeto, como tamanho, complexidade e recursos disponíveis. Esses modelos são geralmente baseados em dados históricos e estatísticas.
- **Estimativa por pontos de função:** esta técnica quantifica o tamanho funcional do software com base nas funcionalidades oferecidas aos usuários. As funcionalidades são categorizadas e pontuadas de acordo com sua complexidade e impacto no sistema, permitindo que os desenvolvedores estimem o esforço com base no tamanho funcional do sistema. A tabela de exemplo da Figura 10 apresenta uma estimativa de esforço utilizando pontos de função (PF) para diferentes funcionalidades de um sistema relacionado ao cliente. Ela serve como uma ferramenta para calcular o esforço necessário em horas, considerando a complexidade de cada funcionalidade, ajudando assim na gestão e no planejamento do projeto.
    - **Cliente-relatório** (tipo SE) possui 6 PF, requer 100 horas de esforço e tem complexidade alta.
    - **Cliente-excluir** (tipo EE) possui 4 PF, requer 30 horas de esforço e tem complexidade média.

- **Cliente-incluir** (tipo EE) possui 4 PF, requer 42 horas de esforço e tem complexidade média.
- **Cliente-consultar** (tipo CE) possui 3 PF, requer 21 horas de esforço e tem complexidade baixa.
- **Cliente-listar** (tipo CE) possui 4 PF, requer 20 horas de esforço e tem complexidade média.
- **Cliente** (tipo ALI) possui 10 PF, requer 100 horas de esforço e tem complexidade alta.

Figura 10 – Exemplo de cálculo de estimativa de pontos de função

FUNCIONALIDADE	TIPO	PF	ESFORÇO EM HORAS	COMPLEXIDADE
Cliente - Relatório	SE	6	100	Alta
Cliente - Excluir	EE	4	30	Média
Cliente - Incluir	EE	4	42	Média
Cliente - Consultar	CE	3	21	Baixa
Cliente - Listar	CE	4	20	Média
Cliente	ALI	10	100	Alta

4.1 Desafios e considerações

Apesar da importância das estimativas de software, elas podem ser desafiadoras devido à incerteza e à complexidade inerentes aos projetos de software. Segundo pesquisadores brasileiros da área de medição de software, como Jorge C. A. de Figueiredo e Hermano P. de Moura (Figueiredo; Moura, 2002), diversos fatores contribuem para essa dificuldade.

Como exemplos, podemos citar:

- **Falta de dados históricos:** em muitos casos, os projetos de software são únicos e não há dados históricos disponíveis para orientar as estimativas. Isso torna difícil prever com precisão o esforço e o tempo necessários para concluir o projeto. De acordo com Figueiredo e Moura (2002), a falta

de dados históricos impacta diretamente na confiabilidade das estimativas, tornando-as mais sujeitas a erros.

- **Mudanças de escopo:** as mudanças frequentes no escopo do projeto podem impactar significativamente as estimativas de software. Alterações nos requisitos ou nas funcionalidades do sistema podem exigir ajustes nas estimativas originais, tornando-as menos confiáveis. Segundo Jacobson et al. (2009), as mudanças de escopo são um dos principais desafios na estimativa de software, uma vez que introduzem incertezas e dificultam a previsibilidade do projeto.
- **Complexidade do projeto:** projetos de software complexos, com requisitos ambíguos ou tecnologias novas, podem ser especialmente difíceis de estimar. A complexidade adicional pode aumentar o risco de erros nas estimativas e resultar em atrasos ou estouros de orçamento. Autores como Rocha e Travassos (2000) destacam que a complexidade do projeto influencia diretamente na precisão das estimativas, sendo necessário adotar técnicas adequadas para lidar com essa complexidade e reduzir a incerteza associada às estimativas.

Os desafios na estimativa de software são multifacetados e exigem abordagens cuidadosas para mitigar seus impactos negativos. A compreensão desses desafios, aliada a técnicas de medição e modelagem adequadas, é essencial para fornecer estimativas mais precisas e confiáveis no desenvolvimento de software.

4.2 Boas práticas para estimativas de software

Além das técnicas específicas de estimativa de software, é fundamental adotar boas práticas para garantir que as estimativas sejam precisas, confiáveis e úteis para o planejamento e a execução do projeto. A seguir, vamos explorar detalhadamente cada uma dessas boas práticas:

- **Coletar dados relevantes:** a coleta de dados relevantes é um passo crucial no processo de estimativa de software. Isso envolve a busca por dados históricos de projetos anteriores semelhantes, tanto dentro da organização quanto em fontes externas, além da utilização desses dados como referência para as estimativas atuais. Os dados históricos podem incluir informações sobre o tamanho do projeto, o esforço de

desenvolvimento, o tempo de conclusão e eventuais desvios do plano original. Ao analisar esses dados, os estimadores podem identificar padrões, tendências e fatores de risco que influenciam as estimativas futuras.

- **Desenvolver modelos realistas:** os modelos de estimativa são ferramentas essenciais para prever o esforço, o tempo e os recursos necessários para concluir um projeto de software. No entanto, é importante que esses modelos sejam realistas e reflitam com precisão a complexidade e os riscos do projeto. Isso significa considerar diversos fatores, como a experiência da equipe, a estabilidade dos requisitos, a disponibilidade de recursos e a incerteza inerente ao processo de desenvolvimento de software. Os modelos de estimativa podem variar em complexidade, desde abordagens simples baseadas em dados históricos até modelos matemáticos mais sofisticados que levam em conta múltiplas variáveis e cenários.
- **Monitorar e atualizar as estimativas:** a estimativa de software não é um processo estático, sendo essencial monitorar e atualizar as estimativas conforme o projeto avança e surgem novas informações. Isso inclui acompanhar o progresso do projeto em relação ao plano original, identificar desvios e ajustar as estimativas conforme necessário. À medida que novos requisitos são adicionados, mudanças são solicitadas ou novos riscos são identificados. Além disso, as estimativas devem ser revisadas e atualizadas para refletir essas mudanças. Além disso, é importante comunicar de forma transparente qualquer alteração nas estimativas para as partes interessadas, garantindo que todos estejam alinhados com as expectativas e o progresso do projeto.

Embora existam desafios associados a esse processo, seguindo boas práticas e utilizando técnicas adequadas, as equipes podem realizar estimativas mais precisas e eficazes, contribuindo para o sucesso do projeto.

TEMA 5 – ANÁLISE DO IMPACTO DAS MÉTRICAS E ESTIMATIVAS NO DESENVOLVIMENTO DE SOFTWARE

As métricas desempenham um papel fundamental no desenvolvimento de software, influenciando diversos aspectos do processo de desenvolvimento e

gerenciamento de projetos. Segundo autores brasileiros reconhecidos na área de medição de software, destacam-se os impactos listados a seguir (Rocha; Travassos, 2000).

5.1 Impacto das métricas no desenvolvimento de software

- **Avaliação da qualidade do software**: as métricas são essenciais para avaliar a qualidade do software ao longo de todo o ciclo de vida do projeto. Weber (2014) ressalta que métricas de qualidade, como taxa de defeitos, cobertura de testes e conformidade com padrões de codificação, fornecem insights valiosos sobre confiabilidade, segurança e manutenibilidade do software. Ao monitorar e analisar essas métricas, as equipes podem identificar áreas de melhoria e tomar medidas corretivas antes que os problemas se tornem mais graves.
- **Gerenciamento de desempenho do sistema:** métricas de desempenho são fundamentais para avaliar o desempenho do sistema e garantir que ele atenda às expectativas de desempenho e escalabilidade (Rocha; Travassos, 2000). Métricas como tempo de resposta, utilização de recursos e *throughput* são frequentemente monitoradas para identificar gargalos de desempenho, otimizar o código e a arquitetura do sistema e garantir uma experiência de usuário satisfatória.
- **Tomada de decisão baseada em dados:** as métricas fornecem uma base objetiva para a tomada de decisões ao longo do ciclo de vida do projeto (Rocha; Travassos, 2000). Ao coletar e analisar dados relevantes, as equipes podem tomar decisões informadas sobre escopo, recursos e prioridades do projeto. Métricas de produtividade, como velocidade de desenvolvimento e taxa de entrega, podem ajudar os gerentes de projeto a avaliarem o progresso do projeto e identificarem possíveis atrasos ou problemas.
- **Melhoria contínua e aprendizado organizacional:** as métricas também desempenham um papel importante na promoção da melhoria contínua e no aprendizado organizacional. Weber (2014) destaca que, ao monitorar e comparar métricas ao longo do tempo, as equipes podem identificar tendências e padrões, aprender com experiências passadas e implementar práticas e processos mais eficazes. Métricas como taxa de

retrabalho e satisfação do cliente podem ajudar as organizações a identificarem áreas de fraqueza e implementarem iniciativas de melhoria.

Ao utilizar métricas de forma eficaz, as equipes podem avaliar a qualidade do software, gerenciar o desempenho do sistema, tomar decisões informadas e promover a melhoria contínua. Ao fazê-lo, podem aumentar as chances de entrega de produtos de alta qualidade, dentro do prazo e do orçamento estabelecidos.

5.2 Impacto das estimativas no desenvolvimento de software

Vamos explorar mais detalhadamente os impactos das estimativas no desenvolvimento de software, incluindo seu funcionamento e suas consequências.

- **Planejamento e orçamento:** as estimativas são fundamentais para o planejamento e o controle de projetos de software, ajudando as equipes a definirem prazos, orçamentos e metas realistas (Boehm, 1986). Estimar de forma precisa o tempo e o esforço necessários para concluir o projeto permite que as equipes aloquem recursos de forma adequada e tomem decisões informadas sobre o escopo e as prioridades do projeto.
- **Gerenciamento de recursos:** as estimativas também influenciam o gerenciamento de recursos, incluindo pessoal, equipamentos e infraestrutura (McConnell, 2006). Com base nas estimativas, os gerentes de projeto podem planejar alocar recursos de forma eficiente, garantindo que as equipes tenham os recursos necessários para concluir o trabalho dentro do prazo e do orçamento definidos.
- **Tomada de decisão:** as estimativas fornecem informações valiosas para a tomada de decisão ao longo do ciclo de vida do projeto (Wiegers, 2003). Elas ajudam as equipes a identificarem riscos potenciais, anteciparem problemas e tomarem medidas proativas para mitigar esses riscos. Além disso, as estimativas permitem que as partes interessadas avaliem o progresso do projeto e façam ajustes conforme necessário para garantir o sucesso final.
- **Comunicação e transparência:** as estimativas também desempenham um papel importante na comunicação e na transparência entre as equipes de desenvolvimento, os gerentes de projeto e as partes interessadas. Ao

fornecer estimativas realistas e atualizadas, as equipes podem estabelecer expectativas claras e manter todas as partes informadas sobre o progresso do projeto e quaisquer desafios ou mudanças que surjam ao longo do caminho.

As métricas e as estimativas são elementos-chave no desenvolvimento de software, mas desempenham papéis diferentes e têm impactos distintos no processo de desenvolvimento. Vamos comparar esses dois aspectos e destacar suas diferenças.

5.2.1 Natureza e propósito

- **Métricas:** as métricas são medidas quantitativas que fornecem informações sobre a qualidade, o desempenho e outros aspectos do software. Elas são utilizadas para avaliar o estado atual do software e identificar áreas de melhoria.
- **Estimativas:** as estimativas, por outro lado, são previsões quantitativas sobre o esforço, o tempo e os recursos necessários para concluir um projeto de software. Elas são utilizadas para planejar e controlar o desenvolvimento do projeto e estabelecer expectativas realistas.

5.2.2 Foco e utilização

- **Métricas**: as métricas são usadas principalmente para monitorar e avaliar a qualidade e o desempenho do software ao longo do ciclo de vida do projeto. Elas ajudam as equipes a identificarem problemas, medirem o progresso e tomarem decisões informadas para melhorar o produto.
- **Estimativas**: as estimativas são usadas principalmente para planejar, orçar e programar o desenvolvimento do software. Elas fornecem uma visão antecipada do tempo e dos recursos necessários para concluir o projeto, sendo fundamentais para o gerenciamento de projetos.

5.2.3 Momento de utilização

- **Métricas:** as métricas são utilizadas ao longo de todo o ciclo de vida do projeto, desde a fase de desenvolvimento até a manutenção do software

em produção. Elas são continuamente coletadas e analisadas para garantir a qualidade e o desempenho do produto.

- **Estimativas:** as estimativas são mais comumente utilizadas no início do projeto, durante a fase de planejamento e definição do escopo. Elas são revisadas e atualizadas conforme o projeto avança, mas têm um foco inicial no estabelecimento de metas e expectativas.

5.2.4 Impacto no processo de desenvolvimento

- **Métricas**: as métricas têm um impacto direto na qualidade e eficácia do software, influenciando a tomada de decisões e as práticas de desenvolvimento. Elas ajudam as equipes a identificarem áreas de melhoria e implementarem processos mais eficientes.
- **Estimativas**: as estimativas têm um impacto direto no planejamento e na execução do projeto, influenciando o cronograma, o orçamento e a alocação de recursos. Elas ajudam a garantir que o projeto seja concluído dentro das restrições estabelecidas.

Embora as métricas e as estimativas sejam distintas em termos de natureza, propósito, momento de utilização e impacto no processo de desenvolvimento, ambas desempenham papéis importantes e complementares no desenvolvimento de software. Enquanto as métricas fornecem insights sobre a qualidade e o desempenho do software, as estimativas ajudam a planejar e controlar o desenvolvimento do projeto. Ao entender e utilizar esses dois aspectos de forma eficaz, as equipes podem aumentar suas chances de sucesso no desenvolvimento de software e na satisfação do cliente, considerando o plano de custos e o cronograma atendido.

FINALIZANDO

Nesta etapa, exploramos os conceitos fundamentais por trás da medição de software, considerando os tipos mais comuns de métricas. Descobrimos que essas métricas não se limitam ao desenvolvimento de software, podendo ser aplicadas em diversos contextos, como gestão de projetos e análise organizacional.

Aprendemos quando e como empregar diferentes métodos de medição de software, e como escolher as métricas mais pertinentes para cada situação.

Exploramos a importância da estimativa de software, compreendendo como o levantamento de esforço pode moldar o planejamento e a execução de projetos de desenvolvimento.

Refletimos se seria possível viver sem medição de software e quais seriam as implicações de tomar decisões sem base em dados concretos e análises precisas.

Ao término desta etapa, almejamos não apenas uma compreensão mais profunda da relevância da medição de software, mas também a aquisição das habilidades necessárias para aplicar esses conceitos de maneira efetiva em nossa prática profissional.

Vamos continuar nossa jornada de aprendizado, aplicando esses conhecimentos e desenvolvendo nossas habilidades para enfrentar os desafios do desenvolvimento de software de forma mais eficiente e eficaz.

REFERÊNCIAS

BOEHM, B. **Software engineering economics**. Prentice-Hall, 1986.

DE SOUZA, G. A. et al. **Utilizando redes neurais para estimar o tamanho de um software**. 2003.

FIGUEIREDO, J. C. A. de; MOURA, H. P. de. Estimating software project effort using analogies. **IEEE Software**, v. 19, n. 6, p. 84-92, 2002.

FREZATTI, F. **Indicadores de desempenho**: os segredos dos gurus. São Paulo: Atlas, 2020.

GUERREIRO, R. **Métricas e indicadores de desempenho:** uma abordagem multidisciplinar. São Paulo: Atlas, 2021.

JACOBSON, I. et al. **Software estimation**: demystifying the black art. Addison-Wesley Professional, 2009.

McCONNELL, S. **Estimating Software Costs**: Bringing Realism to Estimating. Microsoft Press, 2006.

PFLEEGER, S. L. **Engenharia de software**: teoria e prática. São Paulo: Pearson Education, 2016.

PRESSMAN, R. S. **Engenharia de software**: conceitos e práticas. Porto Alegre: AMGH, 2016.

ROCHA, A. R.; TRAVASSOS, G. H. Estimating software project effort using analogies: a comparative case-study-based assessment. **Journal of Systems and Software**, v. 52, n. 1, p. 69-81, 2000.

SANTOS, M. F. Métricas organizacionais: uma abordagem para a avaliação do desempenho empresarial. **Revista Gestão e Produção**, p. 123-138, 2020.

SOMMERVILLE, I. **Engenharia de Software**. São Paulo: Pearson Education, 2016.

VAZQUEZ, C. E. **Análise de pontos de função**: medição, estimativas e gerenciamento de projetos de software. São Paulo: Saraiva Educação SA, 2003.

WEBER, J. **Métricas de software: guia prático**. São Paulo: Editora Exemplo, 2014.

WIEGERS, K. E. **Software Estimation**: Demystifying the Black Art. Microsoft Press, 2003.

YOURDON, E. **Análise Estruturada Moderna**. Porto Alegre: Bookman, 2004.